

SEMANA 3 ACADEMIA MONTERREY

Automatización de pruebas con Selenium y TestNG — Santander México

1. Introducción

El objetivo de esta actividad fue desarrollar pruebas automatizadas utilizando **Selenium WebDriver** y **TestNG**, tomando como sistema bajo prueba el sitio web oficial de **Santander México**.

Las pruebas se enfocaron principalmente en verificar la navegación del sitio y validar que diferentes opciones y enlaces disponibles en el menú principal redirigieran correctamente a las secciones correspondientes.

Para facilitar la organización y mantenimiento del proyecto, se utilizó una estructura basada en el patrón **Page Object Model (POM)**, además de Maven para la administración de dependencias y ejecución de las pruebas.

2. Tecnologías utilizadas

Las principales herramientas utilizadas fueron:

- **Java** — lenguaje utilizado para desarrollar las pruebas.
- **Selenium WebDriver** — automatización e interacción con el navegador.
- **TestNG** — framework utilizado para definir y ejecutar las pruebas.
- **Maven** — administración del proyecto, dependencias y ciclo de construcción.
- **IntelliJ IDEA** — entorno de desarrollo utilizado para implementar y ejecutar el proyecto.
- **Google Chrome** — navegador utilizado para ejecutar las pruebas automatizadas.

- **Selenium Manager** — administración automática del controlador del navegador por parte de Selenium.
-

3. Creación y configuración del proyecto

El proyecto fue creado como un proyecto **Maven**, utilizando la siguiente información:

- **Group ID:** `com.taskflow`
- **Artifact ID:** `selenium-santander-AaronGarza`
- **Version:** `1.0-SNAPSHOT`

Maven permite administrar de manera centralizada las dependencias utilizadas por el proyecto mediante el archivo `pom.xml`.

Entre las dependencias principales se encuentran Selenium y TestNG.

Dependencia de Selenium

```
<dependency>

  <groupId>org.seleniumhq.selenium</groupId>

  <artifactId>selenium-java</artifactId>

  <version>4.48.0</version>

</dependency>
```

Esta dependencia proporciona las clases necesarias para utilizar Selenium WebDriver y controlar el navegador.

Dependencia de TestNG

```
<dependency>
```

```
  <groupId>org.testng</groupId>
```

```
  <artifactId>testng</artifactId>
```

```
  <version>7.9.0</version>
```

```
  <scope>test</scope>
```

```
</dependency>
```

TestNG permite definir los casos de prueba, realizar aserciones, utilizar Data Providers y organizar la ejecución de las pruebas.

4. Estructura del proyecto

El proyecto se organizó utilizando diferentes clases con responsabilidades específicas.

Una estructura simplificada es la siguiente:

selenium-santander-AaronGarza/

|

|— pom.xml

|— testng.xml

|

|— src/

|— main/

```
|  └─ java/
|
|  └─ base/
|    └─ BasePage.java
|
|    |
|    └─ pages/
|      └─ BankHomePage.java
|
|      |
|      └─ factory/
|        └─ DriverFactory.java
|
|
└─ test/
    └─ java/
        └─ tests/
            └─ BankSiteTests.java
```

Además, el proyecto utiliza un listener para la captura de evidencia cuando se presenta un error durante una prueba:

listeners/

```
└─ ScreenshotListener.java
```

5. DriverFactory

La clase `DriverFactory` tiene como responsabilidad la creación y configuración de la instancia de `WebDriver`.

De esta manera, la configuración del navegador se mantiene separada de los casos de prueba.

El concepto general es:

BankSiteTests



DriverFactory



WebDriver



Google Chrome

Esto permite que las pruebas puedan utilizar una instancia configurada de Selenium sin tener que repetir la lógica de creación del navegador en cada prueba.

Durante la configuración actual se utiliza Chrome como navegador.

Además, Selenium Manager permite que Selenium gestione automáticamente el controlador compatible con la versión instalada del navegador, evitando la necesidad de descargar y configurar manualmente un `chromedriver.exe`.

6. Clase BasePage

Se utilizó una clase abstracta denominada `BasePage` como base para las páginas automatizadas.

Su propósito es centralizar operaciones comunes de Selenium que pueden ser reutilizadas por diferentes Page Objects.

Entre las operaciones proporcionadas se encuentran:

- Buscar elementos.
- Esperar a que un elemento sea visible.
- Hacer clic.
- Escribir texto.
- Obtener texto de un elemento.
- Seleccionar opciones de elementos `<select>`.
- Verificar si un elemento está seleccionado.
- Comprobar si un elemento existe.
- Realizar acciones utilizando Selenium.

Por ejemplo, el método `click()` utiliza una espera explícita antes de intentar interactuar con el elemento:

```
protected void click(By locator) {  
  
    WebElement element = wait.until(  
  
        ExpectedConditions.elementToBeClickable(locator)  
  
    );  
  
    element.click();  
  
}
```

El uso de esperas explícitas permite que las pruebas sean menos dependientes de tiempos fijos de carga de la página.

La clase también mantiene una instancia de `WebDriverWait`, configurada con un tiempo máximo de espera, para esperar condiciones específicas antes de continuar con una acción.

7. SantanderHomePage

La clase `BankHomePage` representa la página principal de Santander México dentro del patrón Page Object.

Esta clase hereda de `BasePage` y contiene la lógica específica necesaria para interactuar con los elementos del sitio de Santander.

Entre sus responsabilidades se encuentra la navegación por los diferentes elementos del menú principal.

Por ejemplo, se implementaron métodos para:

- Navegar a una opción del menú.
- Navegar directamente a determinados enlaces.
- Obtener la URL actual.
- Validar elementos relacionados con la navegación.

La separación entre `BasePage` y `BankHomePage` permite distinguir entre:

Funcionalidad genérica de Selenium

`BasePage`

y:

Funcionalidad específica de Santander

`BankHomePage`

Esto facilita el mantenimiento del código y permite reutilizar la funcionalidad común.

8. Diseño de los casos de prueba

Los principales casos de prueba se encuentran en:

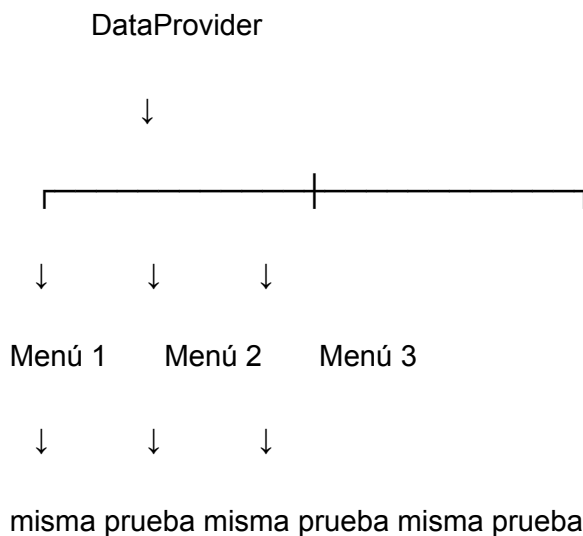
BankSiteTests.java

El objetivo principal de estas pruebas es comprobar que las opciones de navegación del sitio funcionen correctamente.

En lugar de crear un método de prueba completamente independiente para cada enlace, se utilizó un **Data Provider de TestNG**.

Esto permite utilizar el mismo escenario de prueba con diferentes datos.

El concepto utilizado es:



De esta manera, una única prueba puede ejecutarse múltiples veces utilizando diferentes opciones del menú.

Aaron Ulises Garza Rangel

[illegible]

The screenshot displays the 'Test results' tab of the Test Automation Santander application. The top bar indicates '1 suite'. The main content area shows the following results:

- 60 methods, 60 passed
- 0 failed methods (SHOW)

Results

- 60 methods, 60 passed
- Passed methods (hide)
 - ✓ verifyBancaPrivada
 - ✓ verifyHazteCliente
 - ✓ verifySiteLinkSantander(PyMes, pyme/creditos.html, pyme/creditos)
 - ✓ verifySiteLinkSantander(Acerca del Banco, educacion-financiera/, educacion-financiera)
 - ✓ verifySiteLinkSantander(PyMes, alianzas.html, alianzas)
 - ✓ verifySiteLinkSantander(Personas, cuentas/basica/, cuentas/basica)
 - ✓ verifySiteLinkSantander(Personas, creditos-personales/, creditos-personales)
 - ✓ verifySiteLinkSantander(Personas, basica-nomina/, basica-nomina)
 - ✓ verifySiteLinkSantander(Personas, credito-automotriz/, credito-automotriz)
 - ✓ verifySiteLinkSantander(Personas, centro-de-ayuda.html, centro-de-ayuda)
 - ✓ verifySiteLinkSantander(Acerca del Banco, personas/bolsa-de-trabajo.html, bolsa-de-trabajo)
 - ✓ verifySiteLinkSantander(Personas, tutoriales.html, tutoriales)
 - ✓ verifySiteLinkSantander(Personas, select.html, select.html)
 - ✓ verifySiteLinkSantander(Personas, seguros.html#vida, seguros.html#vida)
 - ✓ verifySiteLinkSantander(PyMes, negocio-internacional.html, negocio-internacional)
 - ✓ verifySiteLinkSantander(Personas, colectivos.html, colectivos)
 - ✓ verifySiteLinkSantander(Personas, operaciones-canales-alternos/index.html, canales-alternos)
 - ✓ verifySiteLinkSantander(Acerca del Banco, responsabilidad-social.html, responsabilidad-social)
 - ✓ verifySiteLinkSantander(Personas, sucursales.html, sucursales)
 - ✓ verifySiteLinkSantander(Personas, tarjetas-de-credito/, tarjetas-de-credito)
 - ✓ verifySiteLinkSantander(Personas, #inversiones-a-plazo, inversiones-a-plazo)
 - ✓ verifySiteLinkSantander(Personas, cajeros-automaticos.html, cajeros-automaticos)
 - ✓ verifySiteLinkSantander(Personas, santander-digital/, santander-digital)
 - ✓ verifySiteLinkSantander(Personas, centro-de-seguridad/, centro-de-seguridad)
 - ✓ verifySiteLinkSantander(Personas, promociones/, promociones)

9. Uso de DataProvider

El `DataProvider` contiene la información necesaria para probar diferentes opciones de navegación.

Cada registro proporciona información como:

- Nombre de la opción del menú.
- Parte esperada del enlace.
- Fragmento esperado de la URL.

Conceptualmente, los datos tienen una estructura similar a:

Opción del menú | Enlace esperado | Fragmento esperado de URL

La prueba recibe estos valores como parámetros:

```
@Test(dataProvider = "menuItems")
```

```
public void shouldNavigateToMenuSection(
```

```
    String topMenu,
```

```
    String hrefEndsWith,
```

```
    String expectedUrlFragment) {
```

```
    BankHomePage home = new BankHomePage(driver);
```

```
    home.goToMenuItem(
```

```
        topMenu,  
  
        hrefEndsWith,  
  
        expectedUrlFragment  
  
    );  
  
    Assert.assertTrue(  
  
        home.currentUrl().contains(expectedUrlFragment)  
  
    );  
  
}
```

Esto evita duplicar prácticamente el mismo código para cada una de las opciones disponibles.

Durante la ejecución, TestNG ejecuta el mismo método una vez por cada conjunto de datos proporcionado.

Por este motivo, una sola prueba puede producir muchas ejecuciones individuales dentro del reporte de TestNG.
